

Corrections

- Corrected regarding the same-site property:
same-site does NOT cover port (slides 67-68)

Introduction to Web Technologies (2)

Web Security
Juraj Somorovsky

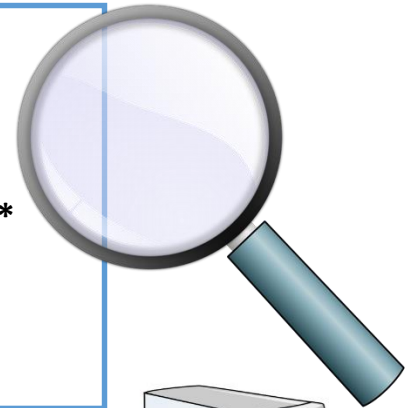
Summer Semester 2024
Paderborn University

Last week: HTTP Header Fields

Human Dog
readable

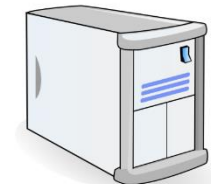


```
GET /History.html HTTP/1.1
Host: www.w3.org
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:66.0)
Gecko/20100101 Firefox/66.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
....
```



Human Dog
readable

```
HTTP/1.1 200 OK
Date: Wed, 15 May 2019 21:54:05 GMT
Last-Modified: Mon, 29 Aug 2016 21:23:30 GMT
ETag: "471d-53b3c79bc1880-gzip"
Accept-Ranges: bytes
Vary: Accept-Encoding
Cache-Control: max-age=21600
Connection: close
...
<html>
....
```



www.w3.org

Hypertext Transfer Protocol

- Message Syntax and Routing ([RFC7230](#))
- Semantics and Content ([RFC7231](#))
- Conditional Requests ([RFC7232](#))
- Range Requests ([RFC7233](#))
- Caching ([RFC7234](#))
- Authentication ([RFC7235](#))

Encodings in Web

- URL Encoding

- Special Characters

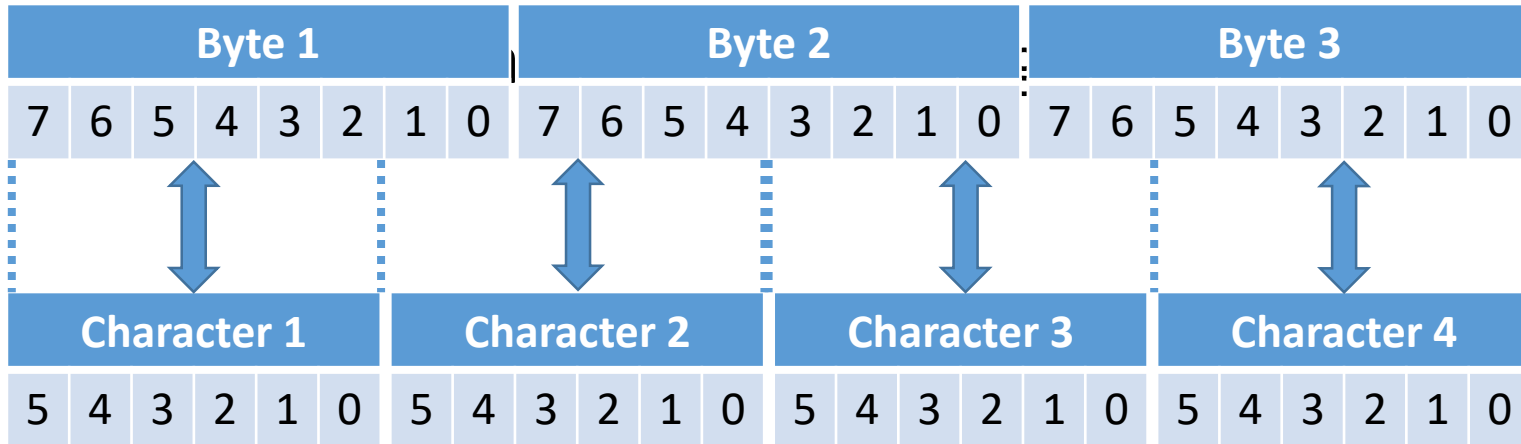
- “;” → %3B
 - “/” → %2F
 - “?” → %3F
 - “:” → %3A
 - “@” → %40
 - “=” → %3D
 - “&” → %26
 - “ ” → %20

- `http://a.com?p1=&!=`
`&p2=value2`

- `http://a.com?p1=%26%3D!%`
`20&p2=value2`

Scan-code	ASCII hex dez	Zeichen	Scan-code	ASCII hex dez	Zch.	Scan-code	ASCII hex dez	Zch.	Scan-code	ASCII hex dez	Zch.
	00 0	NUL ^@		20 32	SP		40 64	@	0D	60 96	`
	01 1	SOH ^A	02	21 33	!	1E	41 65	A	1E	61 97	a
	02 2	STX ^B	03	22 34	"	30	42 66	B	30	62 98	b
	03 3	ETX ^C	29	23 35	#	2E	43 67	C	2E	63 99	c
	04 4	EOT ^D	05	24 36	\$	20	44 68	D	20	64 100	d
	05 5	ENQ ^E	06	25 37	%	12	45 69	E	12	65 101	e
	06 6	ACK ^F	07	26 38	&	21	46 70	F	21	66 102	f
	07 7	BEL ^G	0D	27 39	'	22	47 71	G	22	67 103	g
0E	08 8	BS ^H	09	28 40	(	23	48 72	H	23	68 104	h
0F	09 9	TAB ^I	0A	29 41	)	17	49 73	I	17	69 105	i
	0A 10	LF ^J	1B	2A 42	*	24	4A 74	J	24	6A 106	j
	0B 11	VT ^K	1B	2B 43	+	25	4B 75	K	25	6B 107	k
	0C 12	FF ^L	33	2C 44	,	26	4C 76	L	26	6C 108	l
1C	0D 13	CR ^M	35	2D 45	-	32	4D 77	M	32	6D 109	m
	0E 14	SO ^N	34	2E 46	.	31	4E 78	N	31	6E 110	n
	0F 15	SI ^O	08	2F 47	/	18	4F 79	O	18	6F 111	o
	10 16	DLE ^P	0B	30 48	0	19	50 80	P	19	70 112	p
	11 17	DC1 ^Q	02	31 49	1	10	51 81	Q	10	71 113	q
	12 18	DC2 ^R	03	32 50	2	13	52 82	R	13	72 114	r
	13 19	DC3 ^S	04	33 51	3	1F	53 83	S	1F	73 115	s
	14 20	DC4 ^T	05	34 52	4	14	54 84	T	14	74 116	t
	15 21	NAK ^U	06	35 53	5	16	55 85	U	16	75 117	u
	16 22	SYN ^V	07	36 54	6	2F	56 86	V	2F	76 118	v
	17 23	ETB ^W	08	37 55	7	11	57 87	W	11	77 119	w
	18 24	CAN ^X	09	38 56	8	2D	58 88	X	2D	78 120	x
	19 25	EM ^Y	0A	39 57	9	2C	59 89	Y	2C	79 121	y
	1A 26	SUB ^Z	34	3A 58	:	15	5A 90	Z	15	7A 122	z
01	1B 27	Esc ^[	33	3B 59	;		5B 91	[		7B 123	{
	1C 28	FS ^\	2B	3C 60	<		5C 92	\		7C 124	
	1D 29	GS ^]	0B	3D 61	=		5D 93	]		7D 125	}
	1E 30	RS ^^	2B	3E 62	>	29	5E 94	^		7E 126	~
	1F 31	US ^_	0C	3F 63	?	35	5F 95	_	53	7F 127	DEL

Base64



- $3 \cdot \{0, \dots, 255\} \mapsto 4 \cdot \underbrace{\{A - Z, a - z, 0 - 9, +, /\}}_{64 \text{ different characters}}$

64 different characters

- Data expansion: ~33%
- Padding with one or two "=" characters

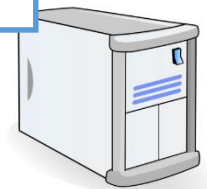
HTTP is stateless

- How to identify users?
- Netscape solution: Cookies



```
POST /authenticate.html HTTP/1.1
Host: www.w3.org

user=nds&password=passwd
....
```



www.w3.org

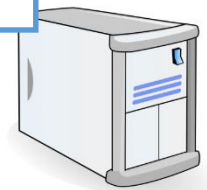
```
HTTP/1.1 200 OK
Set-Cookie: sid=840348032840275520
```

HTTP is stateless

- How to identify users?
- Netscape solution: Cookies



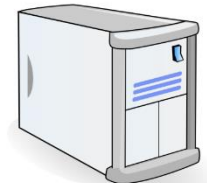
GET /index.html HTTP/1.1
Host: www.w3.org
Cookie: sid=840348032840275520



www.w3.org

HTTP Response Header Fields

```
GET /History.html HTTP/1.1
Host: www.w3.org
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:66.0)
Gecko/20100101 Firefox/66.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
....
```



www.w3.org

Status

Response-Header

Body

```
HTTP/1.1 200 OK
Date: Wed, 15 May 2019 21:54:05 GMT
Last-Modified: Mon, 29 Aug 2016 21:23:30 GMT
ETag: "471d-53b3c79bc1880-gzip"
Accept-Ranges: bytes
Vary: Accept-Encoding
Cache-Control: max-age=21600
Connection: close
...
<html>
....
```



HyperText Markup Language (HTML)

- Markup for creating web sites
- Structure:
 - Header: title, metadata, scripts ...
 - Body: the website

```
<html>
  <head>
...
  </head>
  <body>
...
  </body>
</html>
```

HTML history

- HTML (1993)
 - Hyperlinks, figures,...
- HTML 2.0
 - Forms
- HTML 3.0
 - Toolbars, tables, ...
- HTML 4.0
 - Scripting
- HTML 5
 - SVG, CORS, Web messaging, new security features
 - Constantly developed, living standard with new features

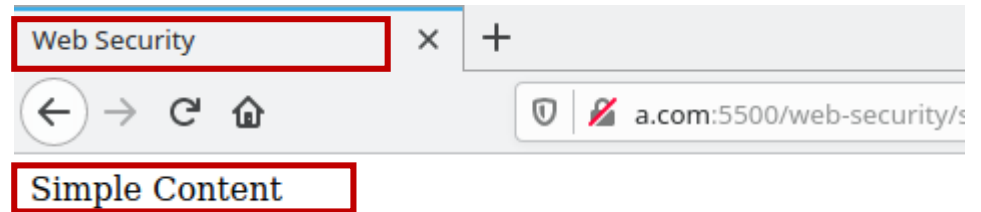
Standard Web Technologies

HTML

- Core Elements: `<head>`, `<title>`, `<body>`
- Formatted Text Content: `<h1>`, ..., `<h6>`, `<b>`, `<i>`, `<p>`
- List: `<ul>`, `<ol>`
- Table: `<table>`
- Form: `<form>`
- Cascading Style Sheets (CSS)
- Dynamic Content (JavaScript)

HTML Example

```
<html>
<head>
  <title>Web Security</title>
</head>
<body>
  Simple Content
</body>
</html>
```



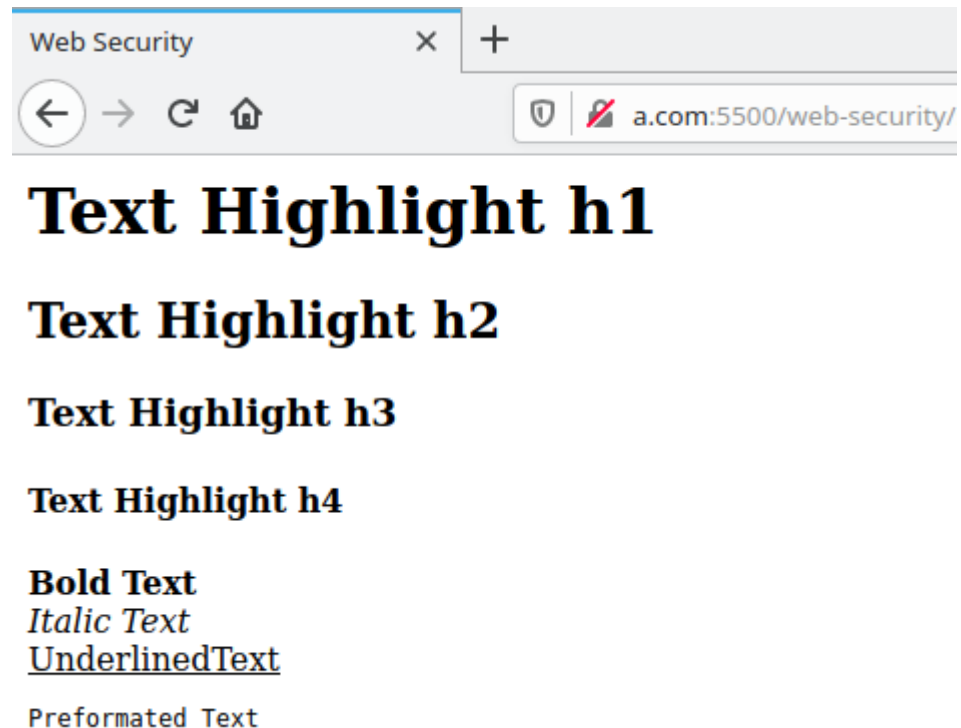
HTML Example: Text Content

```
<html>

<head>
  <title>Web Security</title>
</head>

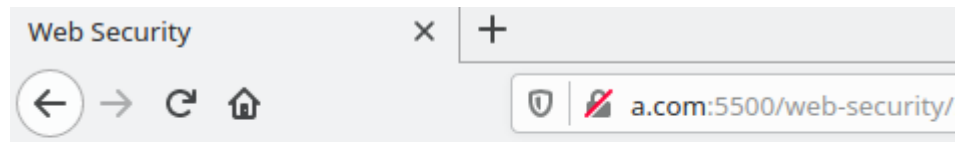
<body>
  <h1>Text Highlight h1</h1>
  <h2>Text Highlight h2</h2>
  <h3>Text Highlight h3</h3>
  <h4>Text Highlight h4</h4>
  <b>Bold Text</b><br />
  <i>Italic Text</i><br />
  <u>Underlined Text</u><br />
  <pre>Preformatted Text</pre>
</body>

</html>
```



HTML Example: Lists

```
<html>
<head>
  <title>Web Security</title>
</head>
<body>
  <h3>An Unordered List</h3>
  <ul>
    <li>Entry 1</li>
    <li>Entry 2</li>
  </ul>
  <h3>An Ordered List</h3>
  <ol>
    <li>Entry 1</li>
    <li>Entry 2</li>
  </ol>
</body>
</html>
```



An Unordered List

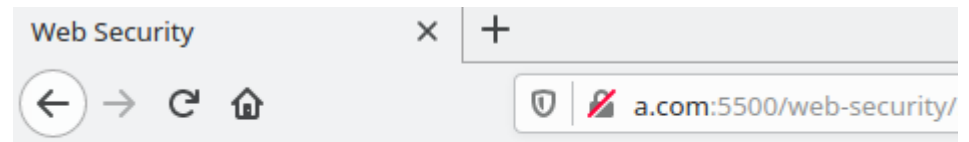
- Entry 1
- Entry 2

An Ordered List

1. Entry 1
2. Entry 2

HTML Example: Tables

```
<html>
<head>
  <title>Web Security</title>
</head>
<body>
  <h3>HTML Table</h3>
  <table border="1">
    <tr>
      <th>Column 1</th>
      <th>Column 2</th>
    </tr>
    <tr>
      <td>Entry 1</td>
      <td>Entry 2</td>
    </tr>
    <tr>
      <td>Entry 3</td>
      <td>Entry 4</td>
    </tr>
  </table>
</body>
</html>
```



HTML Table

Column 1	Column 2
Entry 1	Entry 2
Entry 3	Entry 4

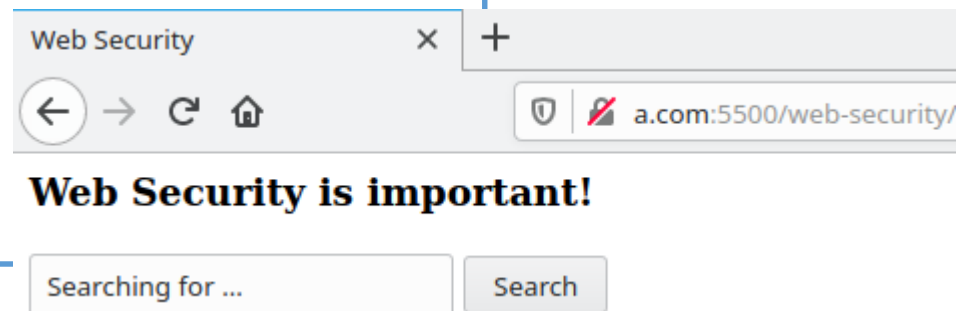
HTML Example: Forms

```
<html>

<head>
  <title>Web Security</title>
</head>

<body>
  <h3>Web Security is important!</h3>
  <form action="#" method="GET">
    <input type="text" name="search" value="Searching for ..." />
    <input type="submit" value="Search" />
  </form>
</body>

</html>
```



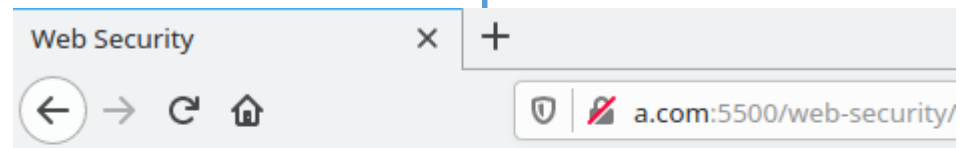
HTML Example: Forms

```
<html>

<head>
  <title>Web Security</title>
</head>

<body>
  <h3>Web Security is important!</h3>
  <form action="#" method="GET">
    <input type="text" name="search" value="Searching for ..." />
    <input type="submit" value="Search" />
  </form>
</body>

</html>
```



<http://a.com:5500/web-security?search=Searching+for+...>

HTML Example: Forms

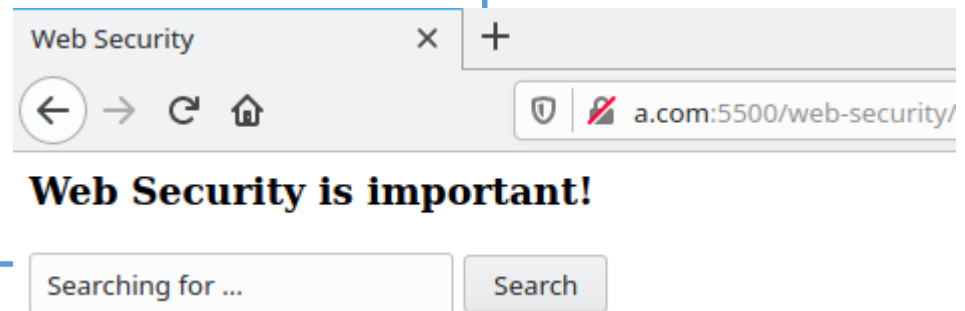
```
<html>

<head>
  <title>Web Security</title>
</head>

<body>
  <h3>Web Security is important!</h3>
  <form action="#" method="POST">
    <input type="text" name="search" value="Searching for ..." />
    <input type="submit" value="Search" />
  </form>
</body>

</html>
```

POST /web-security HTTP /1.1
Host: a.com:5500
...
search=Searching+for+...



Standard Web Technologies

- HTML
 - Core Elements: `<head>`, `<title>`, `<body>`
 - Formatted Text Content: `<h1>`, ..., `<h6>`, `<b>`, `<i>`, `<p>`
 - List: `<ul>`, `<ol>`
 - Table: `<table>`
 - Form: `<form>`

Cascading Style Sheets (CSS)

- Dynamic Content (JavaScript)

Cascading Style Sheets (CSS)

- Goal
 - CSS is a language describing the style of an HTML document
 - CSS describes how HTML elements should be displayed
- Usage
 - External CSS
 - Internal CSS
 - Inline CSS

External CSS

```
<html>

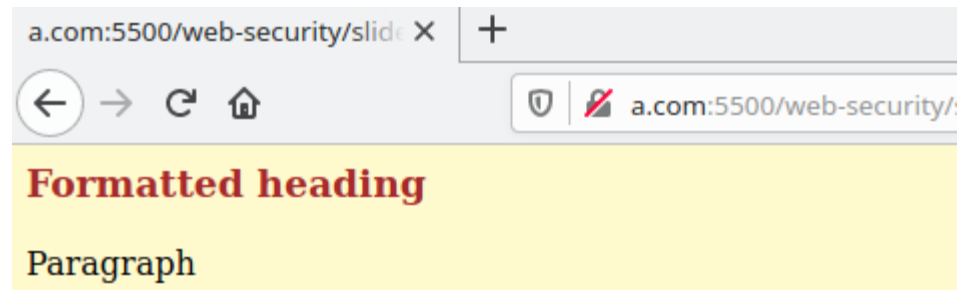
<head>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>

<body>
  <h3>Formatted heading</h3>
  <p>Paragraph</p>
</body>
</html>
```

```
body {
  background-color: lemonchiffon;
}

h3 { color: brown; }
```

styles.css

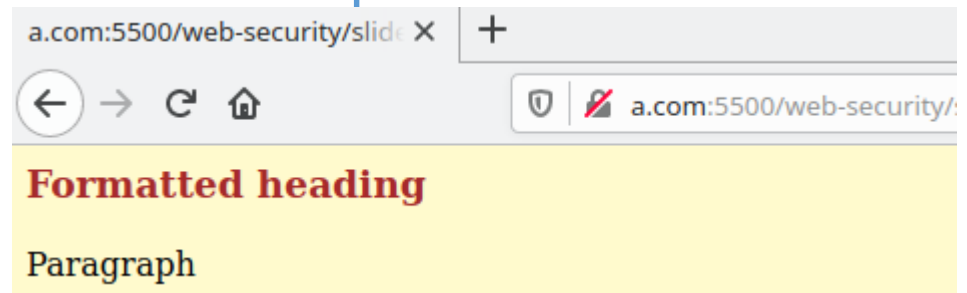


Internal CSS

```
<html>

<head>
  <style>
    body {
      background-color: lemonchiffon;
    }
    h3 {
      color: brown;
    }
  </style>
</head>

<body>
  <h3>Formatted heading</h3>
  <p>Paragraph</p>
</body>
</html>
```

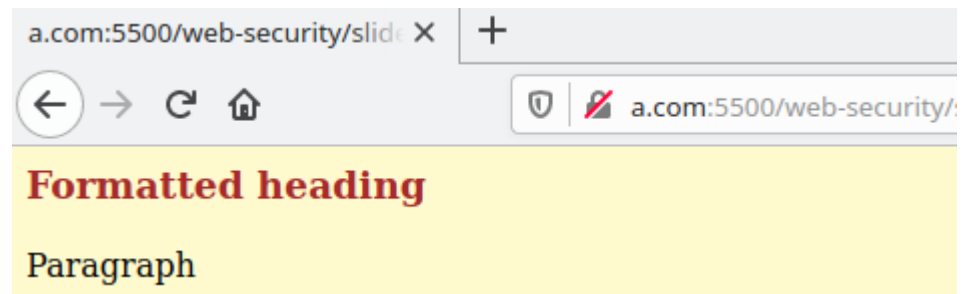


Inline CSS

```
<html>

<head>
</head>

<body style="background-color: lemonchiffon;">
  <h3 style="color: brown;">Formatted heading</h3>
  <p>Paragraph</p>
</body>
</html>
```



DEMO

Questions



404
Not Found

Client-server communication

```
<html>
<head>
  <title>Server Requests</title>
  <link rel="stylesheet" href="https://www.uni-paderborn.de/.../news.min.css" />
</head>
<h1>How many requests are issued to the servers?</h1>
<body>
  
  <iframe src="http://a.com:5500/01_html/1_html-simple.html"></iframe>
</body>
</html>
```

How many GET requests are triggered after visiting this website?

- A) 0
- B) 1
- C) 2
- D) 3 or more

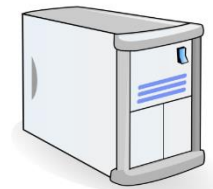


<https://pingo.coactum.de/633763>

Client-server communication



GET /index.html HTTP/1.1



a.com:5500

```
<html>
<head>
  <link rel="stylesheet" href="https://www.uni-paderborn.de/.../news.min.css" />
</head>
<body>
  
  <iframe src="http://a.com:5500/01_html/1_html-simple.html"></iframe>
</body>
</html>
```

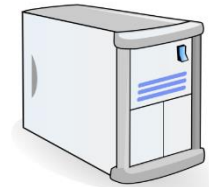
Client-server

```
<html>
<head>
  <link rel="stylesheet" href="https://www.uni-paderborn.de/.../news.min.css"/>
</head>
<body>
  
  <iframe src="http://a.com:5500/01_html/1_html-simple.html"></iframe>
</body>
</html>
```



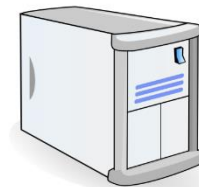
GET /dog.jpeg HTTP/1.1

GET /...simple.html HTTP/1.1



a.com:5500

GET /...news.min.css HTTP/1.1



uni-paderborn.de

Standard Web Technologies

- HTML
 - Core Elements: `<head>`, `<title>`, `<body>`
 - Formatted Text Content: `<h1>`, ..., `<h6>`, `<b>`, `<i>`, `<p>`
 - List: `<ul>`, `<ol>`
 - Table: `<table>`
 - Form: `<form>`
- Cascading Style Sheets (CSS)
- ➡ Dynamic Content (JavaScript)

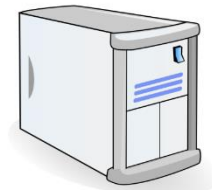
JavaScript

- Script language
- Since HTML 4.0
- Runs in browser
- Dynamic websites
- Handles user inputs, changes document structure,
...

JavaScript



GET /index.html HTTP/1.1



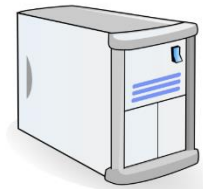
www.w3.org

```
<html>
  <head>
    <script>alert("JavaScript executed!")</script>
  </head>
  <body/>
</html>
```

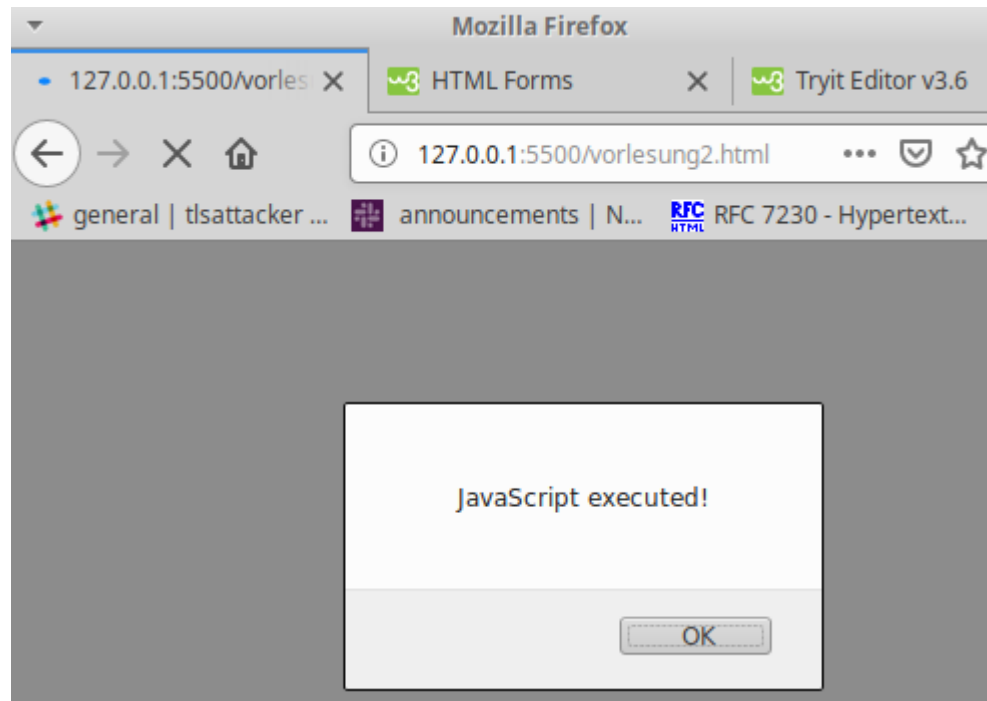

JavaScript



GET /index.html HTTP/1.1



www.w3.org



JavaScript in head

```
<html>
<head>
  <script>
    function myFunction() {
      document.getElementById("demo").innerHTML =
"Paragraph changed.";
    }
  </script>
</head>
<body>
  <h1>JavaScript in Head</h1>
  <p id="demo">A Paragraph.</p>
  <button type="button" onclick="myFunction()">Try
it</button>
</body>
</html>
```

JavaScript in body

```
<html>
<body>
  <h1>JavaScript in Body</h1>
  <p id="demo">A Paragraph.</p>
  <button type="button" onclick="myFunction()">Try
it</button>
  <script>
function myFunction() {
    document.getElementById("demo").innerHTML =
"Paragraph changed.";
}
  </script>
</body>
</html>
```

JavaScript from an external file

```
<html>
<body>
  <h1>External JavaScript</h1>
  <p id="demo">A Paragraph.</p>
  <button type="button" onclick="myFunction()">Try
it</button>
  <script src="myScript.js"></script>
</body>
</html>
```

JavaScript directly in events

```
<html>
<body>
  <h1>What Can JavaScript Do?</h1>
  <p id=„demo">JavaScript can change HTML content.</p>
  <button type=„button“
onclick="document.getElementById('demo').innerHTML='Hello JavaScript!'">
Click Me!</button>

</body>
</html>
```

Events: Connection between JavaScript and HTML

Mouse events

Event Name	Fired When
<code>auxclick</code>	A pointing device button (ANY non-primary button) has been pressed and released on an element.
<code>click</code>	A pointing device button (ANY button; soon to be primary button only) has been pressed and released on an element.
<code>contextmenu</code>	The right button of the mouse is clicked (before the context menu is displayed).
<code>dblclick</code>	A pointing device button is clicked twice on an element.
<code>mousedown</code>	A pointing device button is pressed on an element.
<code>mouseenter</code>	A pointing device is moved onto the element that has the listener attached.
<code>mouseleave</code>	A pointing device is moved off the element that has the listener attached.
<code>mousemove</code>	A pointing device is moved over an element. (Fired continuously as the mouse moves.)
<code>mouseover</code>	A pointing device is moved onto the element that has the listener attached or onto one of its children.
<code>mouseout</code>	A pointing device is moved off the element that has the listener attached or off one of its children.
<code>mouseup</code>	A pointing device button is released over an element.
<code>pointerlockchange</code>	The pointer was locked or released.
<code>pointerlockerror</code>	It was impossible to lock the pointer for technical reasons or because the permission was denied.
<code>select</code>	Some text is being selected.

Events: Connection between JavaScript and HTML

Clipboard events

Event Name	Fired When
<code>cut</code>	The selection has been cut and copied to the clipboard
<code>copy</code>	The selection has been copied to the clipboard
<code>paste</code>	The item from the clipboard has been pasted

Keyboard events

Event Name	Fired When
<code>keydown</code>	ANY key is pressed
<code>keypress</code>	ANY key except Shift, Fn, CapsLock is in pressed position. (Fired continuously.)
<code>keyup</code>	ANY key is released

Questions



404
Not Found

DEMO

Demo



Playing with WebGoat

- Running WebGoat:

```
docker run -it -p 127.0.0.1:8080:8080 -p  
127.0.0.1:9090:9090 webgoat/webgoat
```

- Visit:

<http://localhost:8080/WebGoat/>

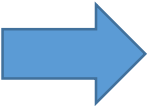
Questions



201

Created

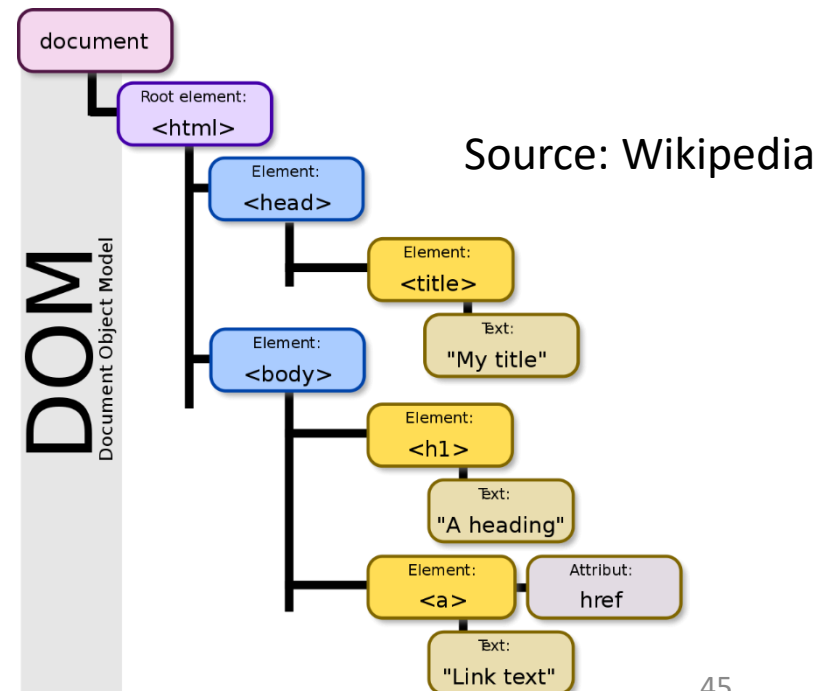
Modern Browser: Overview



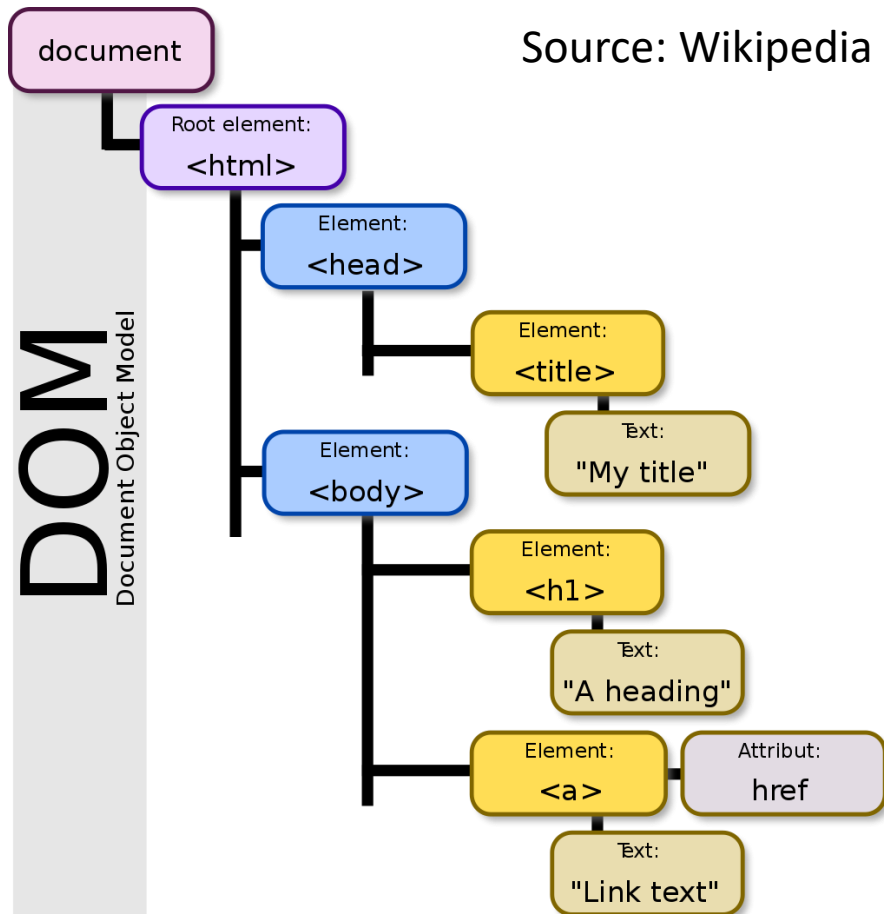
- Document Object Model
- Same-Origin Policy
- Cross-Origin Resource Sharing

Document Object Model

- How to represent HTML, JavaScript, ... internally?
- Platform- and language-neutral interface
- Allows to access objects dynamically, e.g., by using JavaScript
 - Elements, Metadata
 - Using paths, ID attributes



DOM analysis

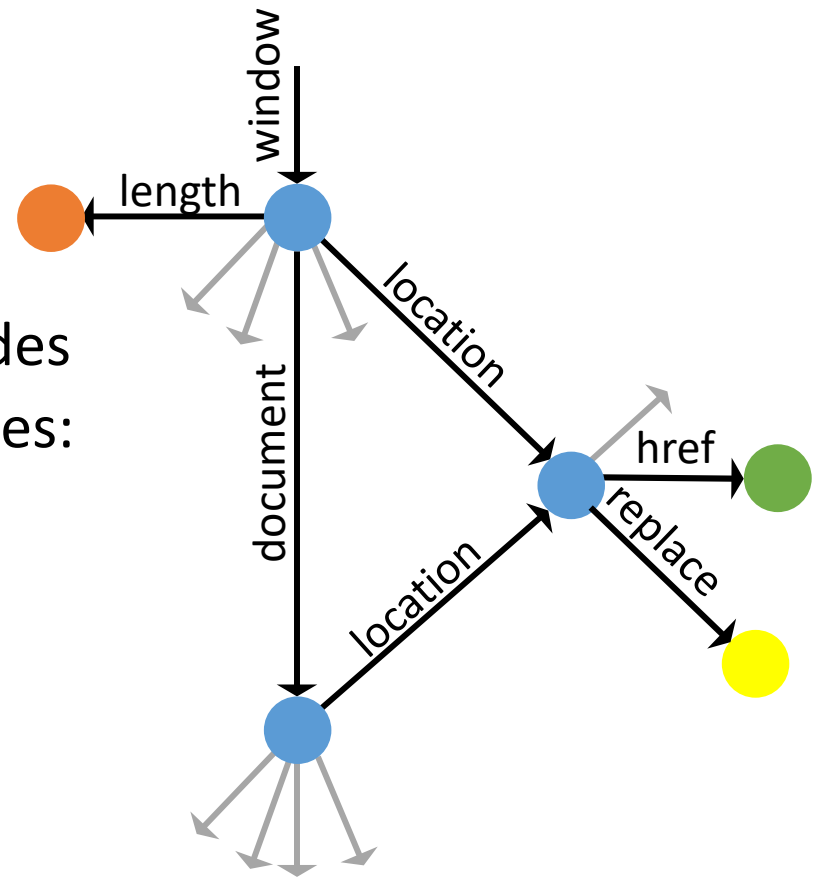


This is very simplified...

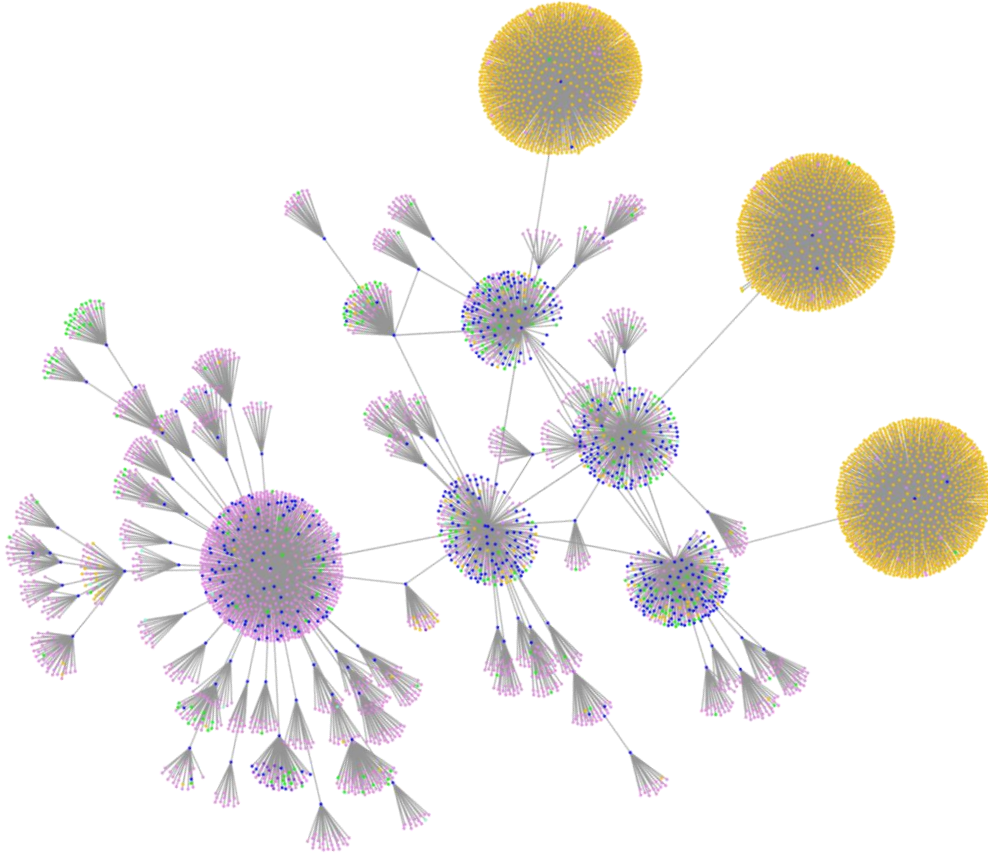
- many elements missing:
 - window
 - location
 - styles

How to analyze the real DOM?

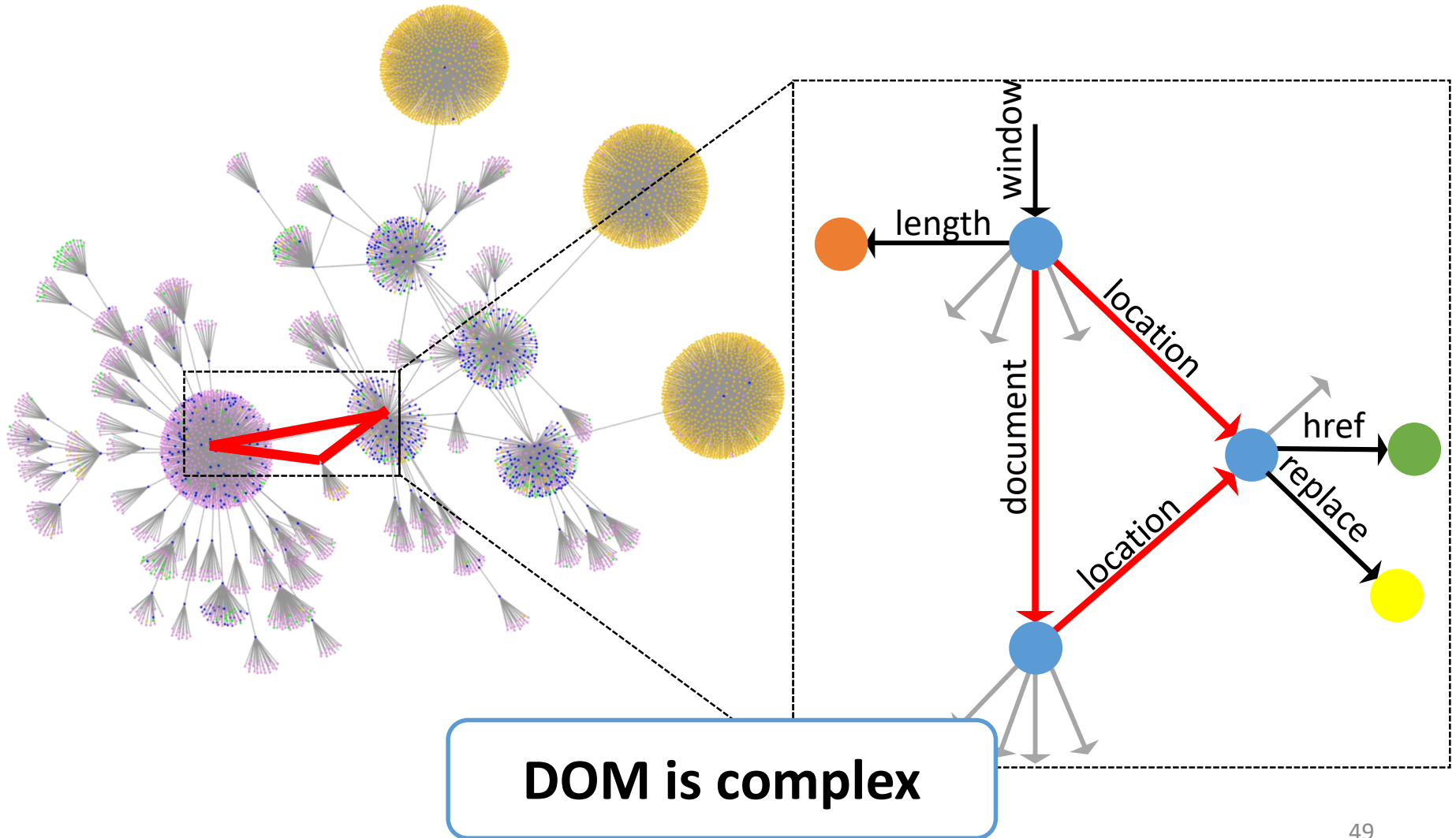
- Work by Dominik Noss
- Methodology:
 - Use JavaScript to traverse nodes
 - Remember and annotate nodes:
 - Strings
 - Functions
 - Objects
 - Numbers
 - Display result



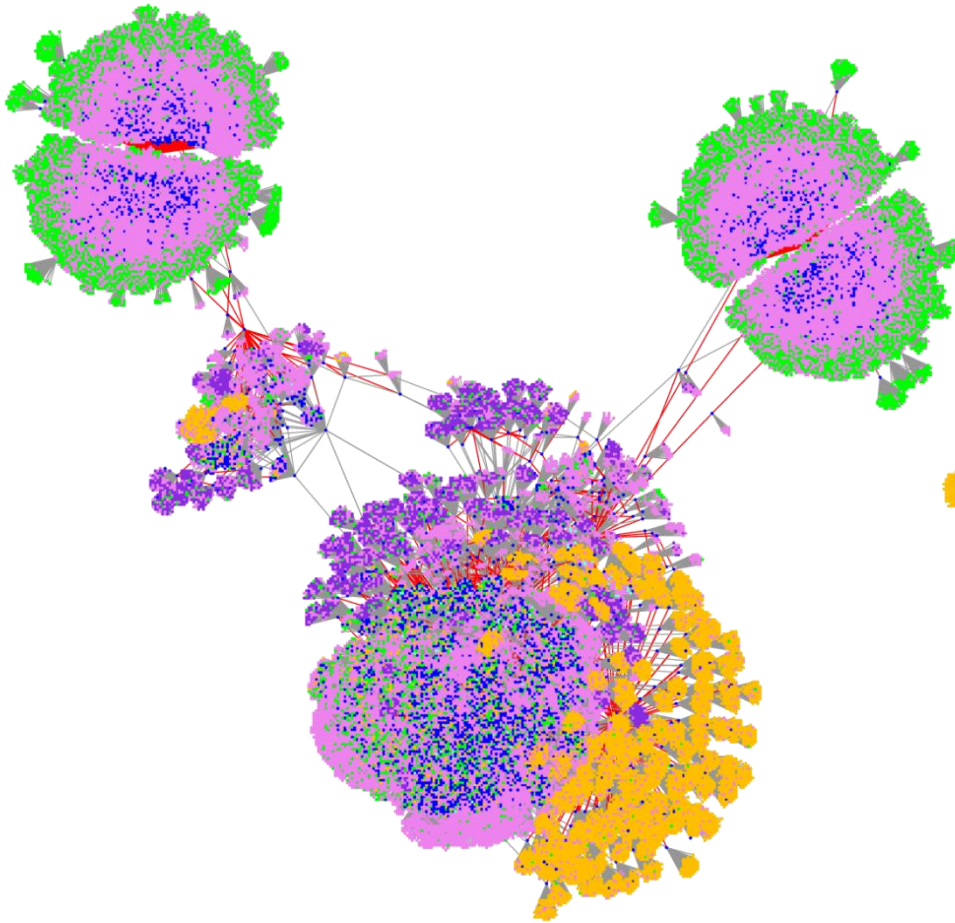
DOM: Minimal page



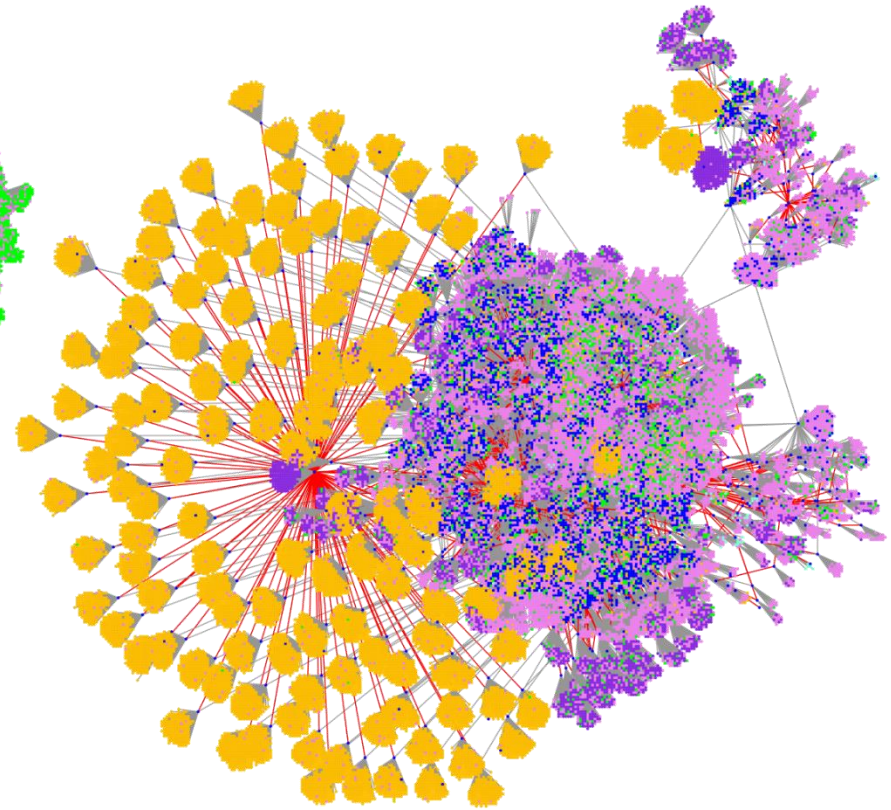
DOM: Minimal page



Different browsers, different results



Firefox 10



Firefox 60

DEMO

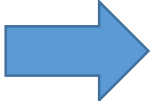
Questions



201

Created

Modern Browser: Overview

- Document Object Model
-  Same-Origin Policy
- Cross-Origin Resource Sharing

Motivation

Bank Website

Login Form

Username:

Password:



LogIn

bank.com

Motivation



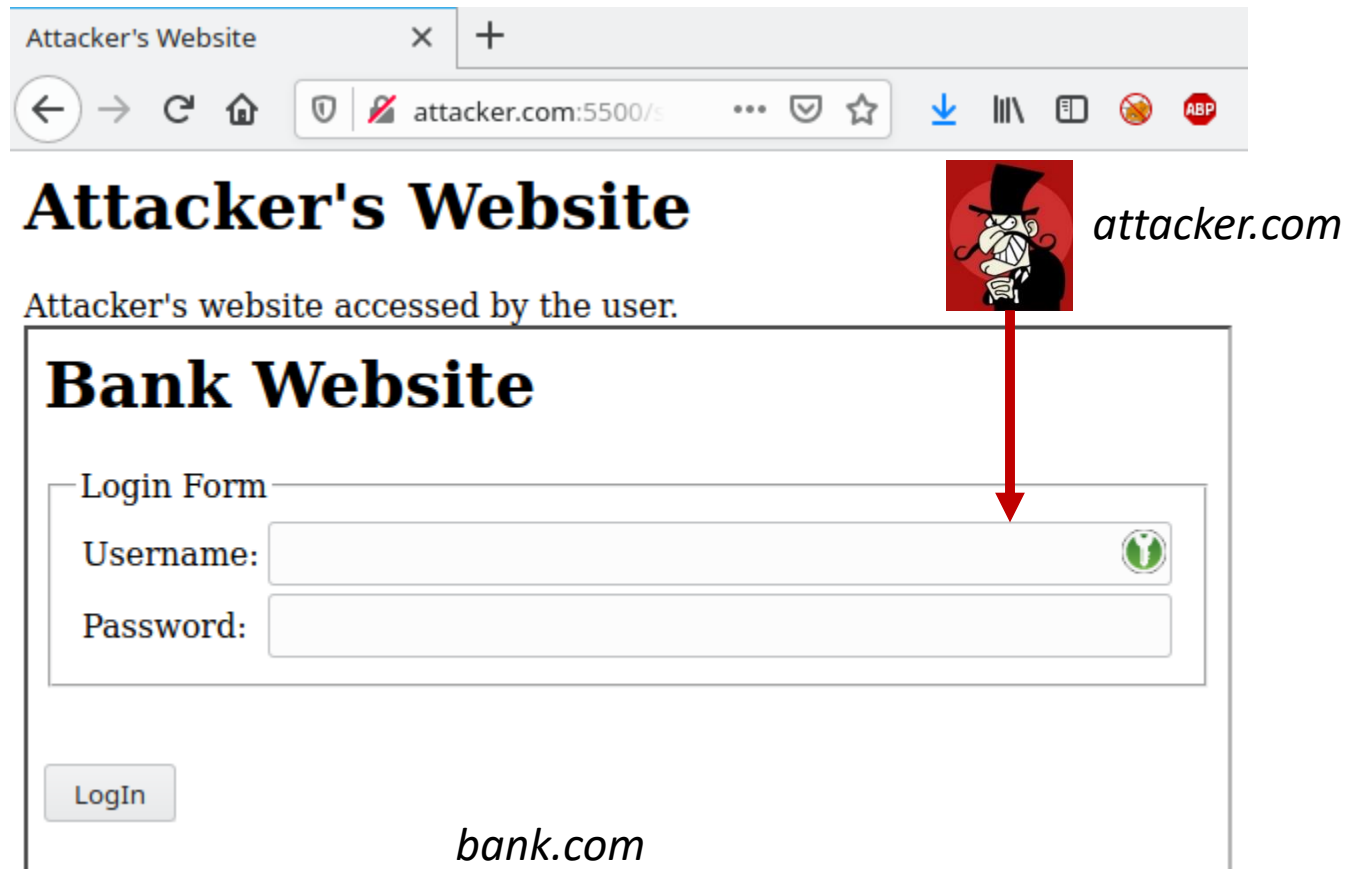
```
<html>
<body>

  <h1>Attacker Website</h1>
  <iframe src="http://bank.com:5500/04_same-origin-
policy/bank-iframe.html"></iframe>

</body>
<body/>
</html>
```

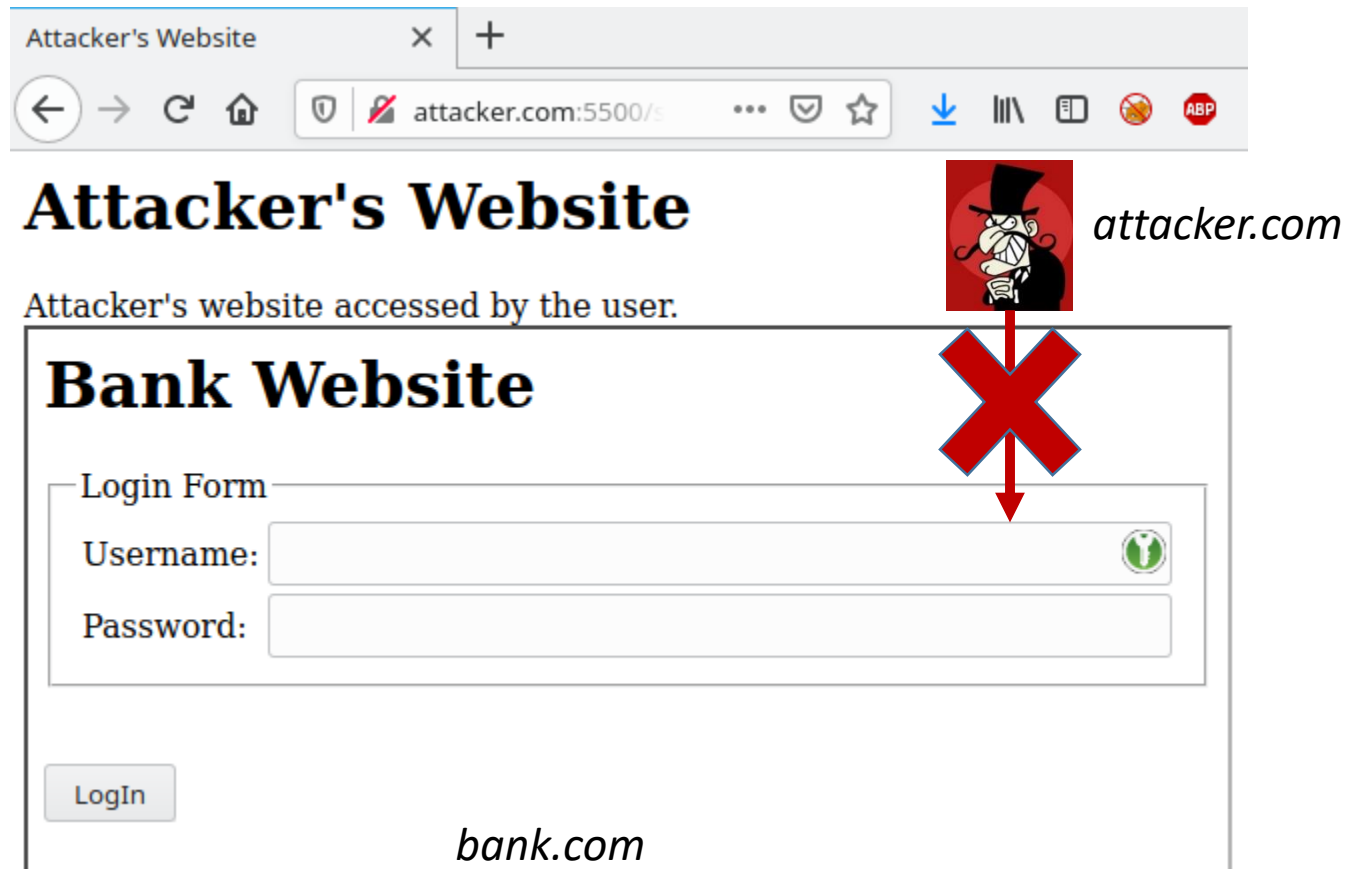
What does this code do?

Motivation



Is this possible?

Same Origin Policy (SOP)



Same Origin Policy (SOP)

- Security concept developed in 1996
- Allows only scripts from the same origin to access DOM
- What is an origin?

Combination of:

- **Protocol**
- **Domain**
- **Port**

SOP examples

<https://a.com>

- 1) <https://a.com/abcdef>
- 2) <http://a.com>
- 3) <https://b.com>
- 4) <https://a.com:443>
- 5) <https://a.com:5443>

Which of these are same origin the above website?

- A) 1
- B) 1, 2
- C) 3
- D) 1, 4
- E) 1, 2, 4, 5



<https://pingo.coactum.de/633763>

SOP examples

https://a.com

https://a.com/abcdef

http://a.com

Incorrect protocol

https://b.com

Incorrect domain

https://a.com:443

https defaults to 443, correct

https://a.com:5443

Different port

Is SOP correctly enforced?

Work by Schwenk, Niemietz, Mainka

Test Setup

- Host Document (HD): HTML document
- Embedded Document (ED): Loaded file (HTML, JPEG, SVG, ...)
- Embedding Element (EE): Embeds ED (e.g., iframe)
- <http://your-sop.com>

Same-site

- Often confused with same-origin
- Same-site is a weaker form of same-origin
 - Checks scheme, eTLD (top level domain)+1

https://www.example.com:443

scheme

eTLD+1

Same-site

- Effective TLDs (eTLDs):
 - .com, .de
 - .co.jp, .github.io

`https://test.github.io:443`

scheme

eTLD+1

- Used for cookies, CSP, and other security related mechanisms
- To be discussed later

Questions



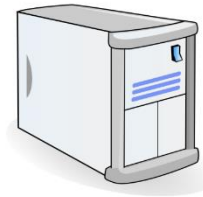
301

Moved Permanently

Accessing data cross-site



GET /index.html HTTP/1.1



website.com

```
<html>
<body>
<div id="extracted"></div>
<script>
  const myDiv = document.getElementById("extracted");
  const xhr = new XMLHttpRequest();
  xhr.open('GET', 'https://website.com');
  xhr.onreadystatechange = function(){
    if (xhr.readyState === 4 && xhr.status === 200) {
      myDiv.innerHTML = xhr.responseText;
    }
  }
  xhr.send();
</script>
</body>
</html>
```

index.html

GET / HTTP/1.1

...



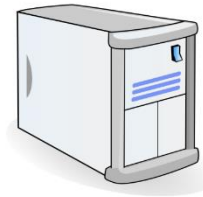
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2020 00:23:53 GMT
Server: Apache/2
Content-Type: text/plain

Some Data

Accessing data cross-site



GET /index.html HTTP/1.1



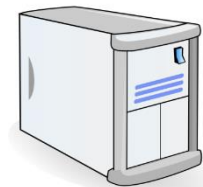
website.com

```
<html>
<body>
<div id="extracted"></div>
<script>
const myDiv = document.getElementById("extracted");
const xhr = new XMLHttpRequest();
xhr.open('GET', 'https://b.server.com');
xhr.onreadystatechange = function(){
  if (xhr.readyState === 4 && xhr.status === 200) {
    myDiv.innerHTML = xhr.responseText;
  }
}
xhr.send();
</script>
</body>
</html>
```

index.html

GET / HTTP/1.1

...



b.server.com

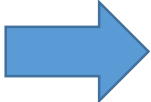
```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2020 00:23:53 GMT
Server: Apache/2
Content-Type: text/plain

Some Data
```

What happens?

Modern Browser: Overview

- Document Object Model
- Same-Origin Policy

 Cross-Origin Resource Sharing

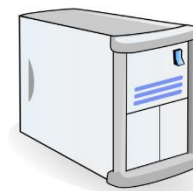
CORS

- The standard adds new HTTP headers that
 - The headers describe which origins are allowed to read the content of a website within the web browser.
- This cross-origin sharing standard can enable cross-site HTTP requests for
 - XMLHttpRequest or Fetch APIs
 - Web Fonts
 - WebGL textures
 - Images/video frames drawn to a canvas using `drawImage()`
 - CSS Shapes from images
- CORS practically "weakens" SOP

CORS



GET /index.html HTTP/1.1



website.com

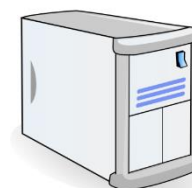
```
<html>
<body>
<div id="extracted"></div>
<script>
const myDiv = document.getElementById("extracted");
const xhr = new XMLHttpRequest();
xhr.open('GET', 'https://b.server.com');
xhr.onreadystatechange = function(){
  if (xhr.readyState === 4 && xhr.status === 200) {
    myDiv.innerHTML = xhr.responseText;
  }
}
xhr.send();
</script>
</body>
</html>
```

index.html

GET / HTTP/1.1

...

Origin: website.com



b.server.com

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2020 00:23:53 GMT
Server: Apache/2
Content-Type: text/plain
Access-Control-Allow-Origin: website.com

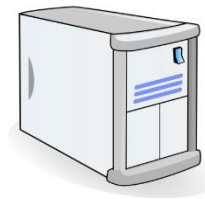
Some Data
```

DEMO

CORS: What could go wrong?



GET /index.html HTTP/1.1



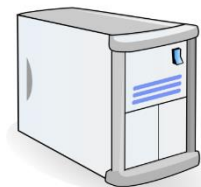
website.com

```
<html>
<body>
<div id="extracted"></div>
<script>
const myDiv = document.getElementById("extracted");
const xhr = new XMLHttpRequest();
xhr.open('GET', 'https://b.server.com');
xhr.onreadystatechange = function(){
  myDiv.innerHTML = this.responseText;
}
xhr.send();
</script>
</body>
</html>
```

index.html

GET /index.html HTTP/1.1

...
Origin: website.com

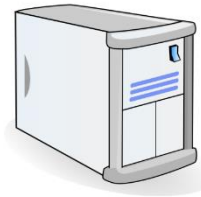


b.server.com

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2020 00:23:53 GMT
Server: Apache/2
Content-Type: text/plain
Access-Control-Allow-Origin: website.com

Some Data

CORS: Origin Reflection



attacker.com

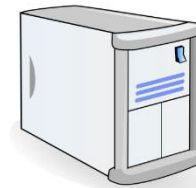
GET /index.html HTTP/1.1



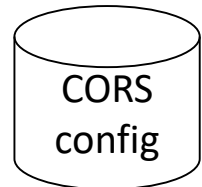
GET /index.html HTTP/1.1

...

Origin: attacker.com



b.server.com



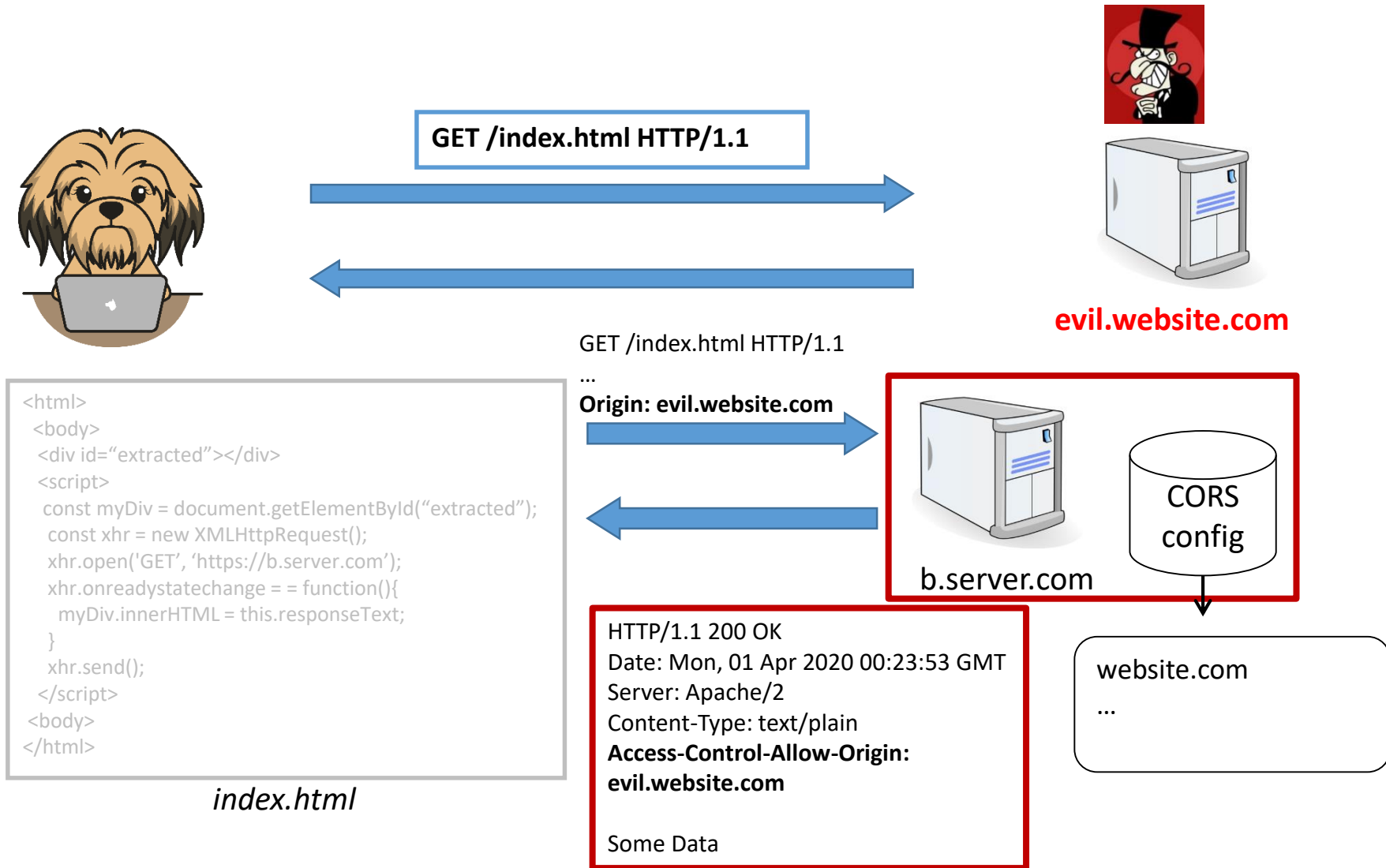
```
<html>
<body>
<div id="extracted"></div>
<script>
const myDiv = document.getElementById("extracted");
const xhr = new XMLHttpRequest();
xhr.open('GET', 'https://b.server.com');
xhr.onreadystatechange = == function(){
  myDiv.innerHTML = this.responseText;
}
xhr.send();
</script>
<body>
</html>
```

index.html

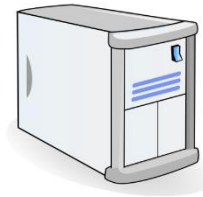
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2020 00:23:53 GMT
Server: Apache/2
Content-Type: text/plain
Access-Control-Allow-Origin: attacker.com

Some Data

CORS: Subdomains



CORS: Post-domain wildcard



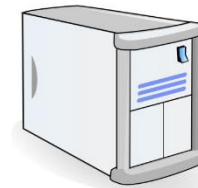
website.com.evil.com



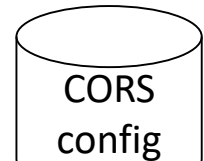
GET /index.html HTTP/1.1

GET /index.html HTTP/1.1

...
Origin: website.com.evil.com



b.server.com



CORS
config

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2020 00:23:53 GMT
Server: Apache/2
Content-Type: text/plain
Access-Control-Allow-Origin:
website.com.evil.com

Some Data

Starts with:
website.com
...
website.com.evil.com

```
<html>
<body>
<div id="extracted"></div>
<script>
const myDiv = document.getElementById("extracted");
const xhr = new XMLHttpRequest();
xhr.open('GET', 'https://b.server.com');
xhr.onreadystatechange = == function(){
  myDiv.innerHTML = this.responseText;
}
xhr.send();
</script>
<body>
</html>
```

index.html

CORS Pitfalls (examples)

Misconfiguration	Description
<i>Post-domain wildcard</i>	domain.com.evil.com is allowed access, can be simply be set up by the attacker
<i>Subdomains allowed</i>	sub.domain.com allowed access, exploitable if the attacker finds XSS in any subdomain
<i>Pre-domain wildcard</i>	notdomain.com is allowed access, which can simply be registered by the attacker
<i>Origin reflection</i>	The origin is simply echoed in ACAO header, any site is allowed to access the resource
<i>Invalid CORS header</i>	Wrong use of wildcard or multiple origins, not a security problem but should be fixed

Conclusions

- Web is complex
- Websites developed from simple HTML sites to complex applications
- First security principles
- Next lecture: attacks (OWASP Top 10)

Questions



301

Moved Permanently

Sources

- Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing [RFC7230](#)
- Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content ([RFC7231](#))
- Hypertext Transfer Protocol (HTTP/1.1): Conditional Requests ([RFC7232](#))
- Hypertext Transfer Protocol (HTTP/1.1): Range Requests ([RFC7233](#))
- Hypertext Transfer Protocol (HTTP/1.1): Caching ([RFC7234](#))
- Hypertext Transfer Protocol (HTTP/1.1): Authentication ([RFC7235](#))
- <https://developer.mozilla.org/de/docs/Web/HTTP/Methods>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

Acknowledgements

- Ömmes pictures by Janine.Plickert@rub.de



- Tessa pictures by https://www.fiverr.com/tnhs_project



- This is collaborative work with Juraj Somorovsky, Tibor Jager, Andreas Mayer, Christian Mainka, Jörg Schwenk, Marcus Brinkmann, and Vladislav Mladenov